EMBEDDED LINUX
SENSOR MONITORING SYSTEM

Assignment 02 | Write Embedded Application

TECHNICAL REPORT

Student: Nguyen Thanh Phong

Student ID: SE182514

Target: ARM Cortex-A9 (QEMU Versatile Express)

Host: Ubuntu 26.04.1 LTS / VirtualBox

Report date: 21 October 2026

Evidence basis: uploaded terminal screenshots and measurement logs. Source files were not included in the evidence ZIP; implementation details are described at architectural level and should be verified against the final submitted source tree.

Abstract

This report describes a multithreaded Embedded Linux Sensor Monitoring System (SMS) targeting an ARM Cortex-A9 system emulated with QEMU. The design integrates a character-device-style kernel sensor driver, a bounded shared circular buffer, POSIX message queues, four worker threads, threshold-based alerts, and file logging. The project also explores resource correctness, runtime debugging, performance profiling, and scheduling wake-up latency.

The supplied evidence shows an ARM functional run with matching sample production, processing and logging counts; Valgrind reported zero memory errors and no outstanding allocations during a 30-second host mock run. A 60-second host mock run processed 120 samples with zero buffer overflows. strace exposed very frequent message-queue receive calls, which were substantially reduced after introducing a 10 ms backoff on EAGAIN. Scheduling measurements include 1,000 wake-up observations for each of SCHED_OTHER and SCHED_FIFO under an emulated ARM environment.

Scope and evidence boundaries

The report distinguishes direct measurements from design descriptions. The uploaded archive includes 22 screenshots plus test and profiling logs, but not the C source tree, Makefiles, ARM binaries or kernel module. Conclusions therefore describe what these artifacts demonstrate; source-level assertions or fully exhaustive error-path validation would require additional review.

Contents

- 1. Goals and acceptance criteria
- 2. Development and target environment
- 3. System architecture
- 4. Task 1 - Kernel driver and multithreaded application
- 5. Task 1 - Functional validation
- 6. Task 2 - Memory correctness and resource usage
- 7. Task 3 - GDB debugging
- 8. Task 3 - strace and optimization
- 9. Task 3 - perf profiling
- 10. Task 4 - Scheduling experiment design
- 11. Task 4 - Results and interpretation
- 12. Problems, limitations, lessons learned
- 13. Conclusion, references and evidence inventory

1. Goals and acceptance criteria

The principal engineering goal is to combine Linux kernel and user-space programming in a sensor-monitoring pipeline. The test plan covers functional processing and observability as well as memory use and scheduling behavior.

Area	Observable outcome
Kernel integration	Load sensor module, access /dev/sms_sensor, configure interval using ioctl, inspect /proc/sms_stats.
Concurrency	Execute Sensor, Processor, Alert and Logger threads with bounded inter-thread communication.
Monitoring	Apply temperature / humidity thresholds and route alerts without losing processed records.
Reliability	Check Ctrl+C/shutdown, message queue cleanup, file logging, and buffer overflows.
Diagnostics	Use Valgrind, /proc, GDB, strace, and perf.
Scheduling	Compare SCHED_OTHER with SCHED_FIFO priority 50 using >=1,000 wake-ups.

Metrics and definitions

Sample consistency means produced = consumed = processed = logged for a particular completed run. An alert-delivery consistency check compares generated and received alerts. This does not by itself prove that every possible malformed input, device error, or queue saturation path is handled.

Wake-up lateness is measured as actual CLOCK_MONOTONIC wake-up time minus the absolute scheduled deadline, clamped to zero if early. It is useful as a scheduler-related latency metric but differs from the interval-to-interval jitter of a physical sensor. All interpretations in the report use this definition.

2. Development and target environment

Component	Configuration
Host OS	Ubuntu 26.04.1 LTS (Resolute Raccoon)
Host kernel	7.0.0-38-generic
Virtualization	VirtualBox host virtual machine
Emulated board	QEMU vexpress-a9 / ARM Cortex-A9, SMP 2, 512 MB
Target kernel	Linux 5.15.0, customized for POSIX message queues and high-resolution timers
Toolchain	arm-linux-gnueabihf-gcc 15.2.0
Debugging	GDB 17.1; Valgrind 3.26.0; strace; perf 7.0.14

Build and execution model

The target uses a Linux zImage, Versatile Express device tree, and an initramfs image. The kernel module is loaded after boot; message queues are mounted under /dev/mqueue. The user-space executable then runs with a configured sampling interval, alert thresholds and optional duration.

A separate x86-64 mock build feeds synthetic sensor observations to the same downstream processing pipeline. It enables host-based memory checking and profiling but does not exercise kernel-driver device I/O. Therefore, ARM device results and host mock results are intentionally reported separately.

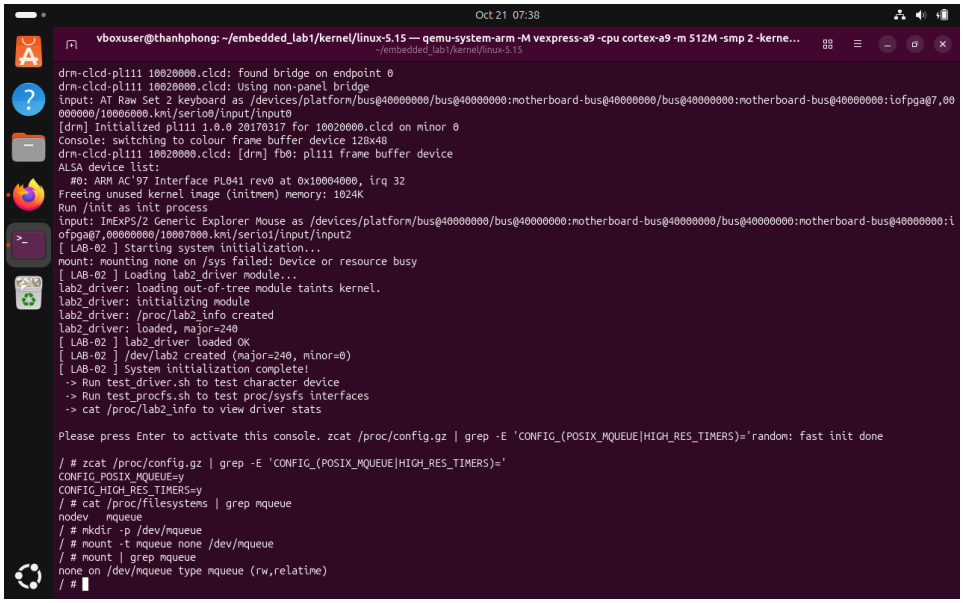


Figure 1. Kernel configuration / message-queue validation evidence.

3. System architecture

The software is organized as a producer/consumer pipeline with isolation between sampling, processing, alert output and durable logging. The central bounded queue decouples acquisition timing from downstream work; POSIX message queues decouple processing from the alert and logging consumers.

Kernel sensor driver -> /dev/sms_sensor -> Sensor Thread

Sensor Thread -> Circular Buffer (64 entries) -> Processor Thread

Processor Thread -> Alert Message Queue -> Alert Thread

Processor Thread -> Log Message Queue -> Logger Thread

Thread responsibilities

Thread	Main responsibility	Shared resource
Sensor	Read and parse a sensor observation; enqueue it on an absolute-time schedule.	Character device; circular buffer
Processor	Dequeue, compute rolling mean over up to ten observations; test thresholds.	Circular buffer; two message queues
Alert	Receive alert events and print threshold violations.	Alert message queue
Logger	Append timestamp / temperature / humidity to log; periodically flush and rotate.	Log message queue; file

The documented configuration uses a 64-entry circular buffer, message queues for the output consumers, a 10-observation moving average, and a 1 MB log-rotation threshold. Rotation behavior is part of the planned implementation, but the supplied short-run logs report zero rotations; rotation correctness has not been demonstrated by this evidence set.

4. Task 1 - Kernel driver integration

The project uses a loadable sensor driver exposing /dev/sms_sensor and /proc/sms_stats. Sensor data is formatted as timestamp, temperature and humidity, with an ioctl operation for acquisition-interval configuration. A reader may receive EAGAIN when a read is attempted before the next permitted sample time.

The ARM user-space sensor path reopens the device between observations to reset the file offset used by this implementation. The acquisition loop combines CLOCK_MONOTONIC deadlines with clock_nanosleep(TIMER_ABSTIME), reducing accumulated sleep drift relative to purely relative delays.

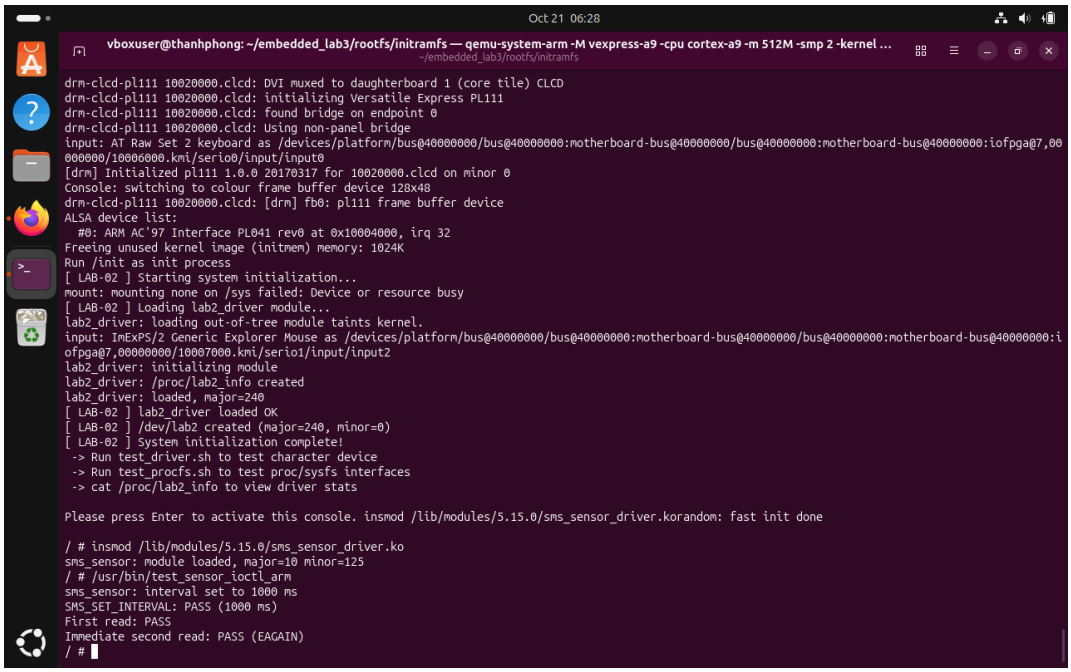


Figure 2. ioctl / device access test from the ARM execution environment.

Real-time primitives

The primary application attempts SCHED_FIFO priority 50 for the acquisition thread and locks the process address space with mlockall where supported. On the Ubuntu host mock build, real-time scheduling may fail with EPERM under standard privileges; the ARM QEMU environment was used for real-time-mode demonstrations.

4.1 Task 1 - Data flow and failure handling

The circular buffer provides finite capacity instead of unbounded memory growth. The processing thread forwards observations to the logger queue and conditionally emits alert messages. The final counters are useful lightweight invariants for detecting data loss during a completed run.

Path	Expected behavior	Evidence caveat
Sensor -> buffer	Successful observation is enqueued; full buffer reports overflow.	No overflows shown in attached test runs.
Processor -> logger	Log message sent for each processed sample.	Matching totals in host and ARM logs.
Processor -> alert	Emit threshold/spike alerts, received by Alert Thread.	Generated equals received in cited runs.
Shutdown	Join workers; close/unlink POSIX queues; release buffer.	Empty /dev/mqueue reported in interactive checks.

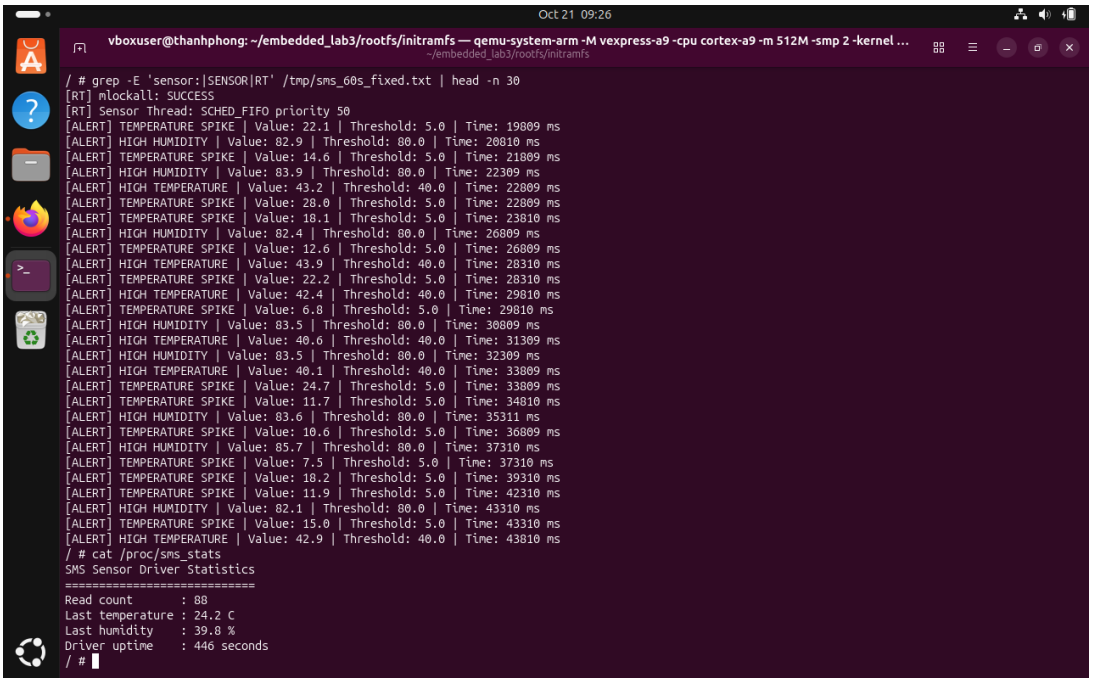


Figure 3. Representative sensor-thread execution and diagnostic output.

A 10-second post-optimization ARM re-test was reported interactively with 15 samples produced, consumed, processed and logged; nine alerts were generated and received. The attached archive does not include a separate text log for that exact short run, so the main attached screenshots and test records remain the primary documentary evidence.

5. Task 1 - Functional validation

A longer ARM demonstration captured successful message processing and alert delivery. The screenshots document end-to-end activity on the emulated target, whereas the host mock offers deterministic synthetic input for repeatability.

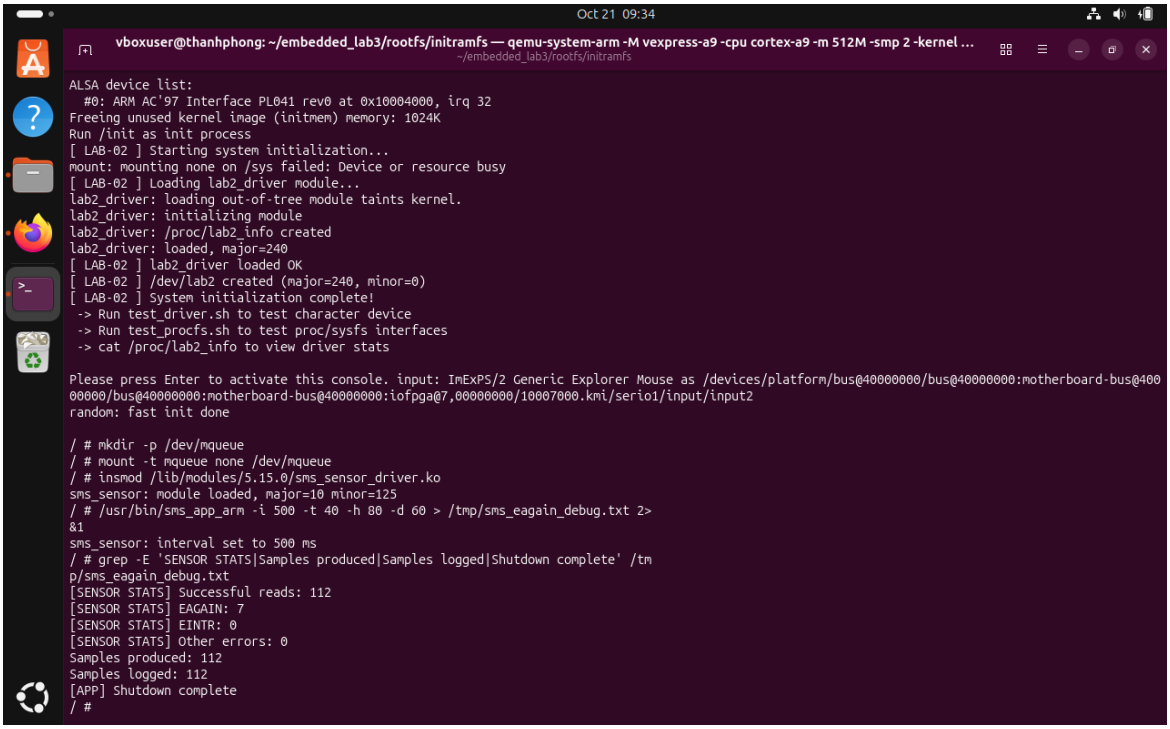


Figure 4. ARM application end-to-end functional test screenshot.

An earlier 60-second target run was reported with 112 successful device reads and seven EAGAIN responses, with no buffer overflows. That is within the previously used acceptance window of 120 +/- 10 successful reads at a 500 ms nominal period, but timing is not perfectly deterministic under nested virtualization.

Functional confidence

The strongest evidenced claims are successful end-to-end operation in selected completed runs and consistent counters. Stress tests for message-queue saturation, repeated log rotation, device disappearance, and every cleanup failure branch are still outstanding and should not be counted as proven.

6. Task 2 - Valgrind memory correctness

A 30-second host mock execution was profiled with Valgrind Memcheck. This test covers the application's mock-sensor and downstream processing allocation lifecycle, not the Linux kernel driver.

Valgrind metric	Recorded value
Command	./sms_app_mock -i 500 -t 40 -h 80 -d 30
Allocations / frees	10 allocs / 10 frees
Total heap allocated	8,238 bytes
Live heap at exit	0 bytes in 0 blocks
Errors	0 from 0 contexts
Result	No leaks detected in this run

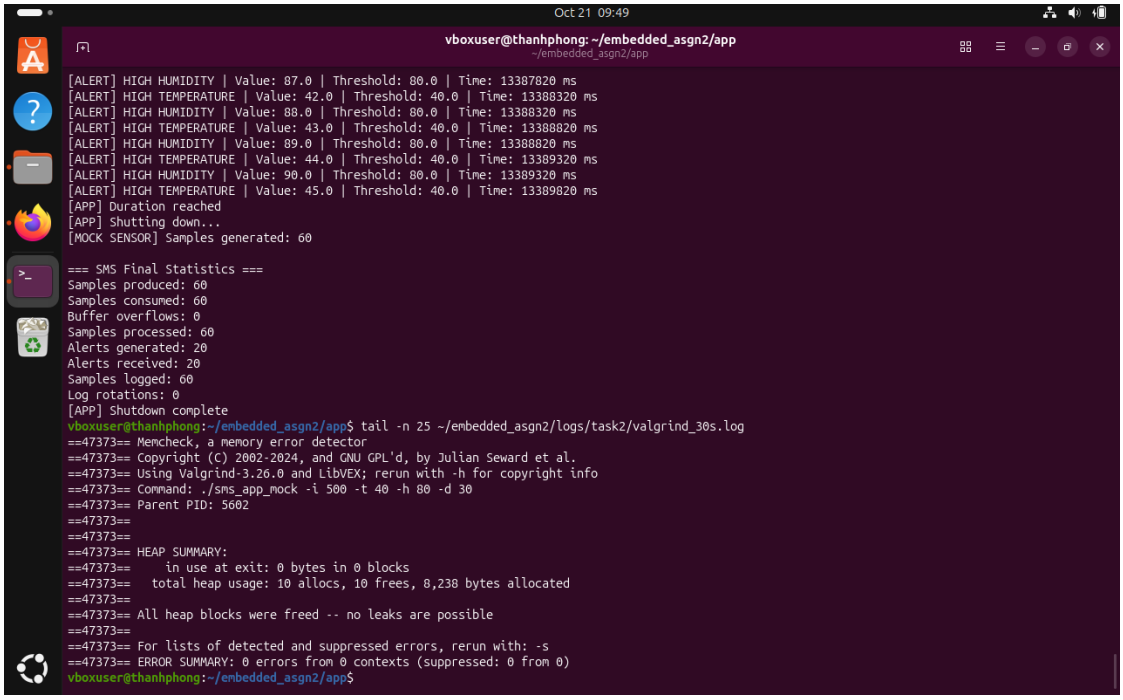


Figure 5. Memcheck summary: all allocated blocks freed, zero reported errors.

The absence of leaks for this duration is a valuable test result but is not a formal proof of leak freedom across all inputs, error paths, longer runs or the target driver.

6.1 Task 2 - Process memory and maps

Elapsed time	VmRSS	VmSize	VmPeak	Threads
5 s	2,128 kB	101,104 kB	166,640 kB	5
30 s	2,128 kB	101,104 kB	166,640 kB	5
55 s	2,128 kB	101,104 kB	166,640 kB	5

The resident set size stayed constant at 2,128 kB across the three observations. VmSize and the five-thread count were also unchanged. This supports memory stability over the measured interval, although three snapshots cannot exclude brief transient peaks.

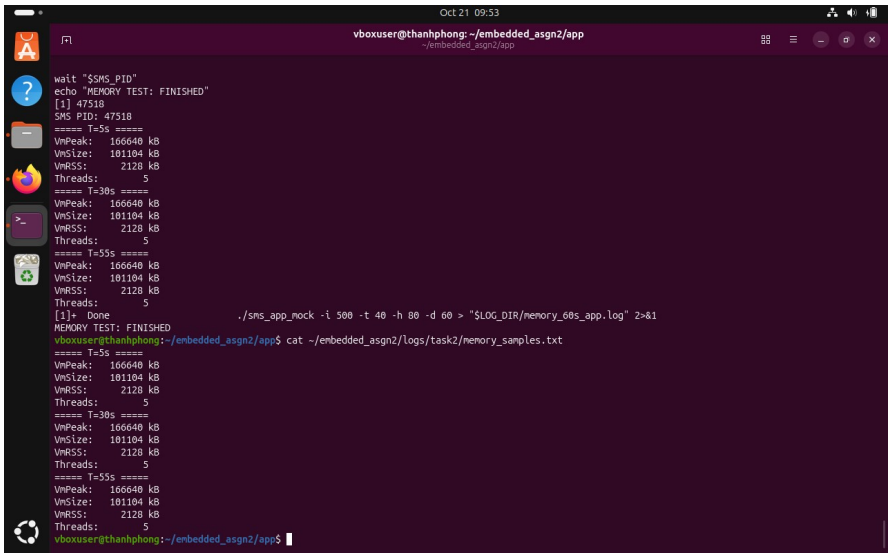


Figure 6. /proc/PID/status samples at 5, 30 and 55 seconds.

Mapped memory regions

The captured /proc/PID/maps includes executable mappings, a labeled [heap], a labeled [stack], libc.so.6, the dynamic loader and anonymous mappings. Anonymous ranges cannot all be assigned unique purposes without additional tracing.

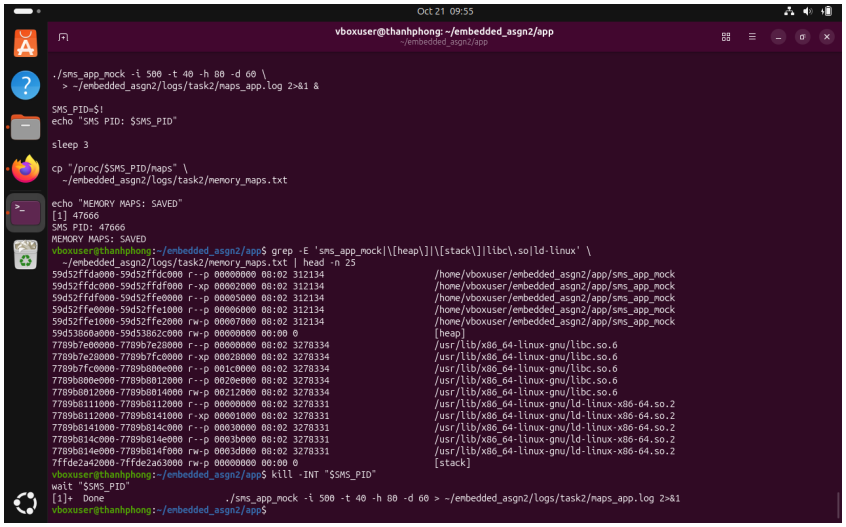


Figure 7. Executable, shared-library, stack and related mapped regions.

6.2 Task 2 - 60-second host mock run

Counter	Value
Samples generated	120
Samples produced / consumed	120 / 120
Samples processed / logged	120 / 120
Alerts generated / received	40 / 40
Buffer overflows	0
Log rotations	0
Exit	Shutdown complete

A 500 ms nominal sampling period produced exactly 120 mock observations in the recorded 60-second test. The logging and processing totals match the production total, and alert transport was lossless in this run. Empty POSIX message-queue state after execution was observed in an interactive terminal check.

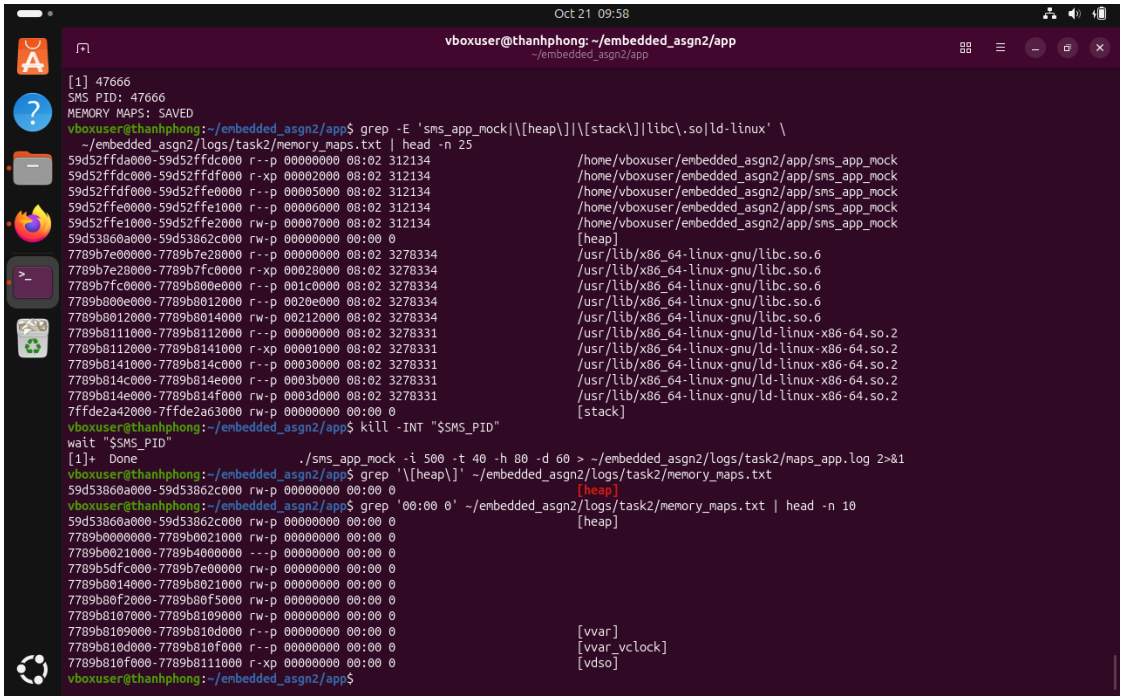


Figure 8. Heap and anonymous memory mappings used for process-memory analysis.

Resource-handling caveat

References to pthread_join, mq_close, mq_unlink and circular_buffer_destroy were found in the application's cleanup path during interactive inspection. The evidence does not demonstrate complete branch coverage for startup failures or exhausted file descriptors.

7. Task 3 - GDB multithreaded debugging

The x86-64 mock executable was built with debug symbols and examined using GDB 17.1. Stopping the process during execution showed five live threads: the main thread and the four worker threads. Thread-specific backtraces identified application functions such as processor_thread_func, logger_thread_func and alert_thread_func.

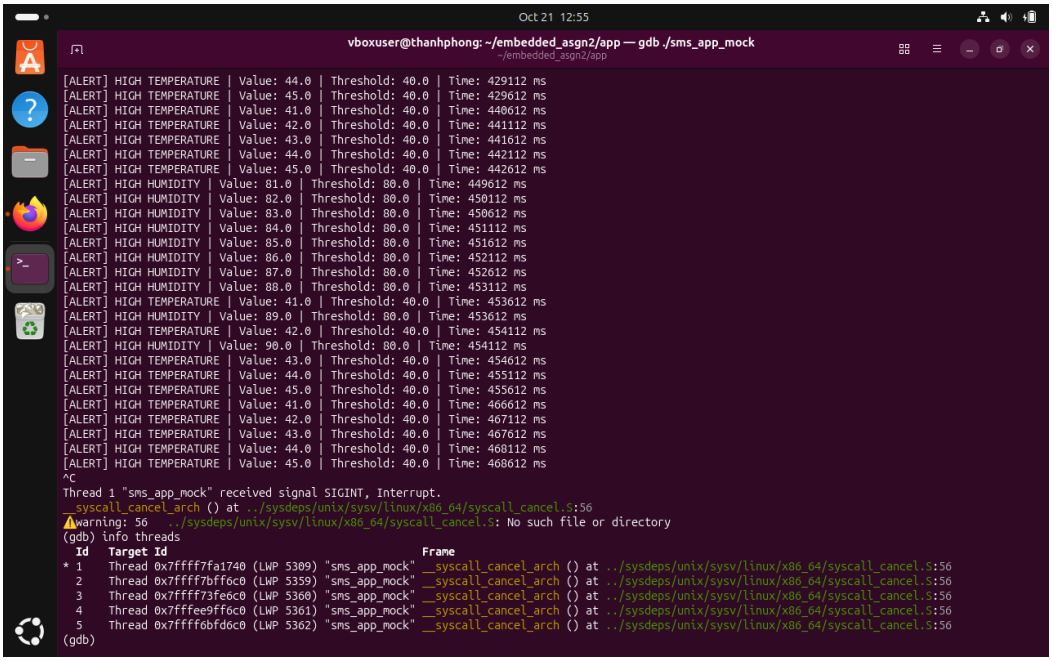


Figure 9. GDB info threads: five active threads.

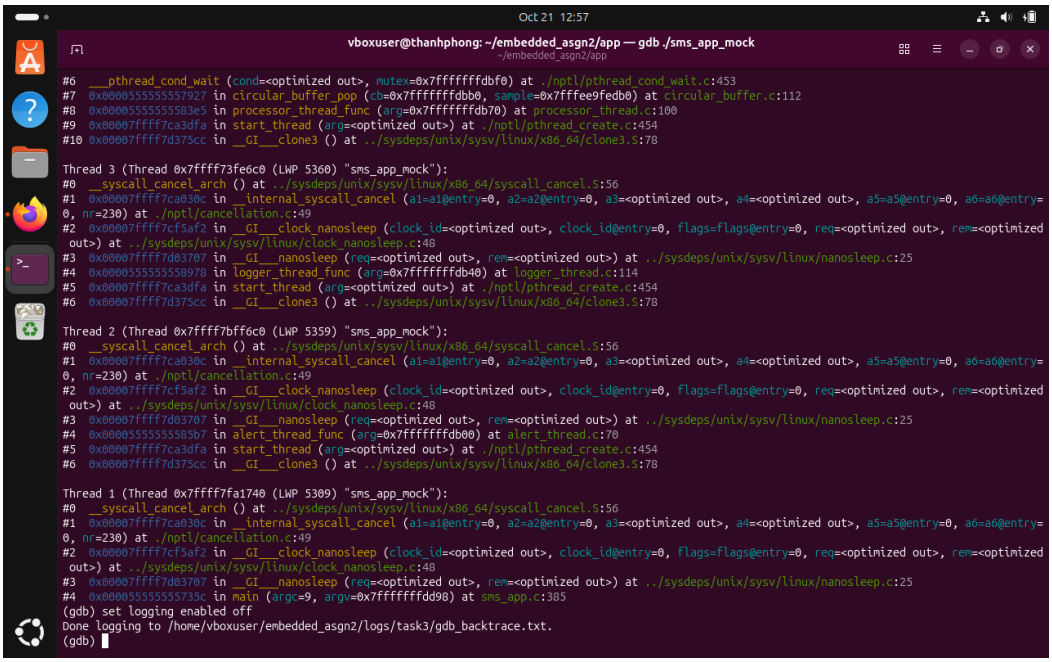


Figure 10. Captured thread backtraces and function call stacks.

7.1 Task 3 - Breakpoint and local variables

A breakpoint was placed at processor_thread.c:105, immediately before rolling_average_get. The debugger observed a mock sample with temperature 20 and humidity 40. After stepping past the function call, temp_avg = 20 and hum_avg = 40. These are consistent with the first observation entering an initially empty averaging window.

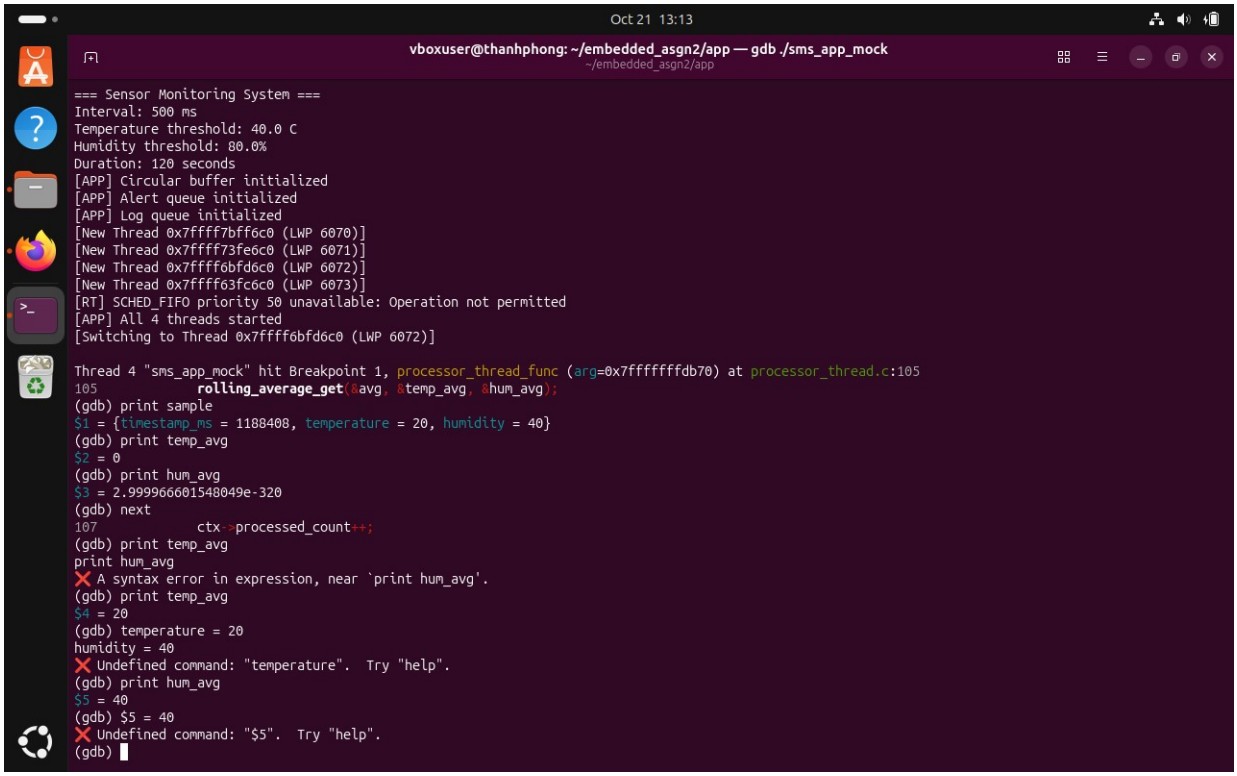


Figure 11. Processor breakpoint and inspected sample / rolling-average variables.

Debugging lessons

Locals inspected before their first assignment may contain stale or indeterminate data and should not be interpreted as valid sensor statistics. The stepping sequence is important: read the sample, execute the averaging routine, then inspect the output variables.

When a process is stopped in a library system call, a missing local copy of libc assembly source is not evidence of application failure. Application frames are located using backtraces and relevant debug symbols.

8. Task 3 - strace syscall analysis

strace -c -f collected per-system-call statistics across the host mock threads for a 30-second run. The dominant system-call time was associated with sleeps, futex synchronization, and message-queue receive attempts.

Call (baseline)	Calls	Reported syscall time share
clock_nanosleep	350	50.17%
futex	249	25.59%
mq_timedreceive	1,752,935	24.23%
write	104	<0.01%
mq_timedsend	79	<0.01%

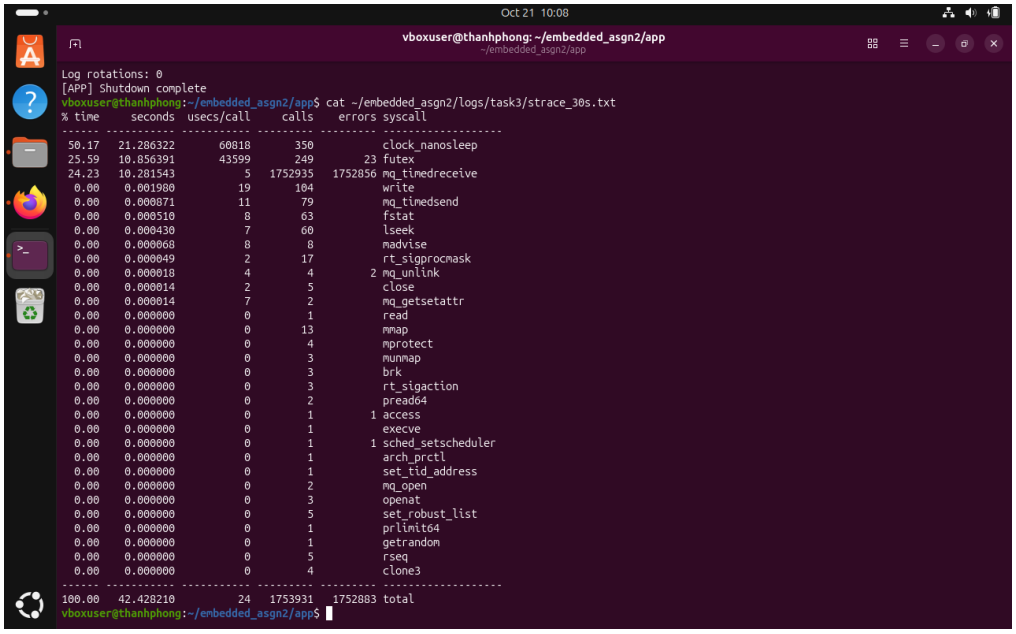


Figure 12. Baseline strace aggregated system-call statistics.

The large count of mq_timedreceive attempts, 1,752,856 of which were reported as errors, revealed an avoidable polling pattern. POSIX message queues had been opened with O_NONBLOCK, so empty reads returned EAGAIN immediately even though the receiving code supplied an absolute timeout.

8.1 Task 3 - Optimization and comparison

A targeted fix introduced a 10 ms nanosleep backoff after receiving EAGAIN in Alert and Logger loops while preserving nonblocking queue mode. Preserving O_NONBLOCK on shared descriptors avoids inadvertently making the Processor Thread block when a queue becomes full.

Metric	Before backoff	After backoff
mq_timedreceive calls	1,752,935	5,533
mq_timedreceive errors	1,752,856	5,454
Time recorded for receive syscalls	10.281543 s	0.038000 s
Relative reduction in receive calls	-	99.684%

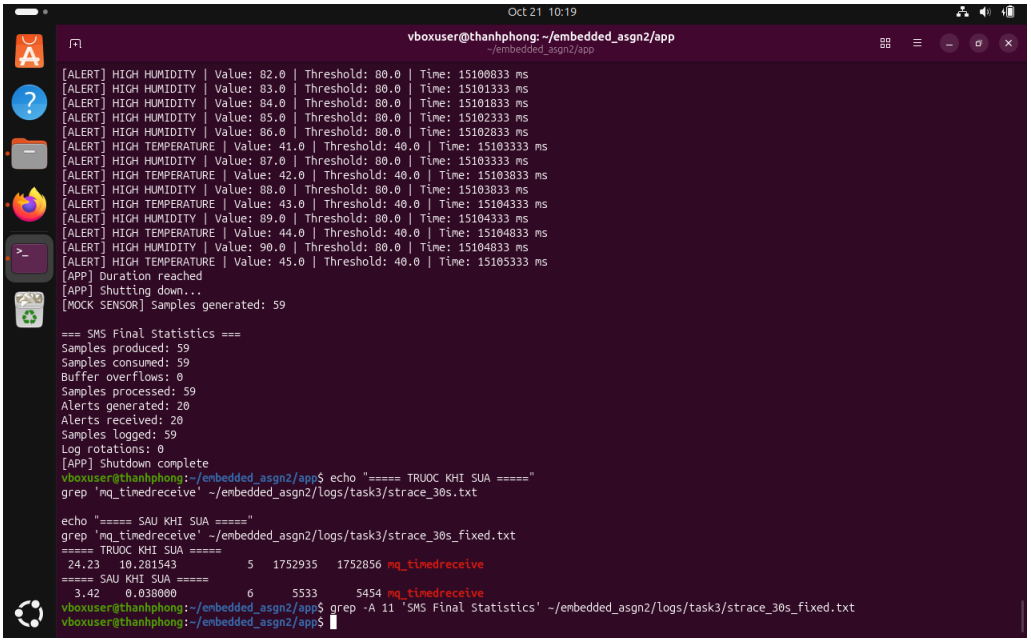


Figure 13. Before-and-after mq_timedreceive call-count comparison.

This comparison establishes a large reduction in unnecessary system-call traffic. It does not isolate CPU improvements under identical system load, and a 10 ms polling backoff can introduce additional queue-consumer latency. A blocking receive with independently managed queue-descriptor flags or notification primitives may be preferable for a production implementation.

9. Task 3 - perf profiling

On Ubuntu inside VirtualBox, perf first failed due to host PMU support and the perf_event_paranoid permission setting. A temporary policy adjustment enabled software event measurement. The record/report step then provided only three cpu-clock samples, which is insufficient to draw a robust hot-function ranking.

Measured item	Value
perf stat duration	30.011398630 s
task-clock	155.47 ms
context switches	3,917
page faults	80
Approximate single-core equivalent	0.52%
perf record samples	3; zero lost

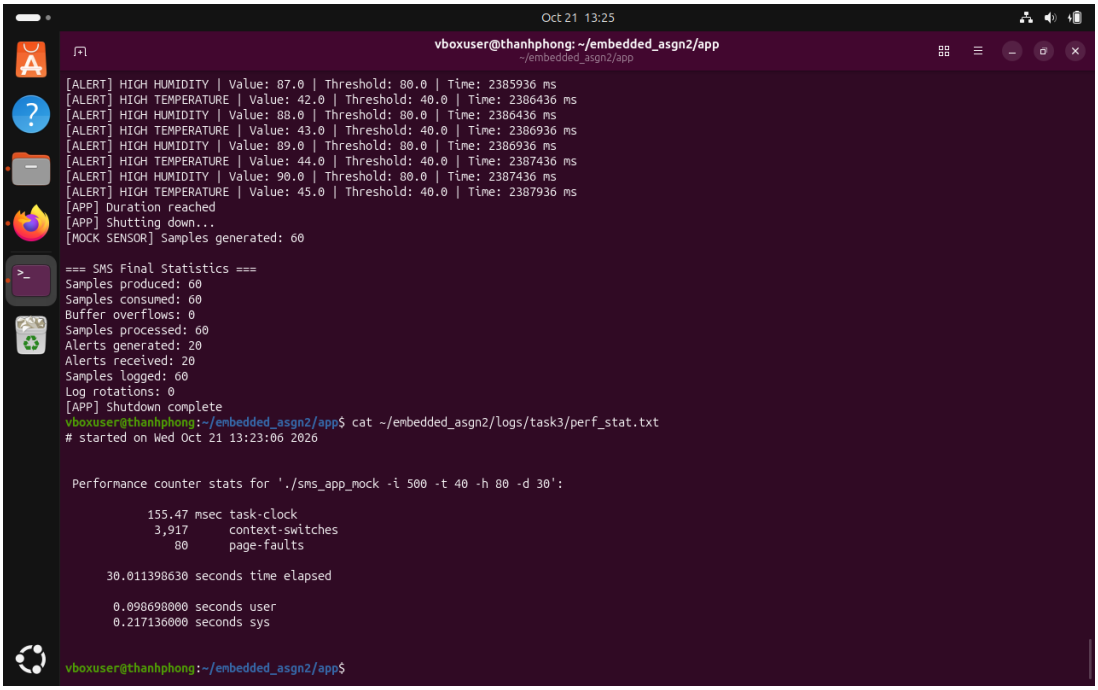


Figure 14. perf stat showing software-event counters for the 30-second mock run.

The approximate CPU fraction is task-clock / elapsed wall time = 0.15547 / 30.0114 = 0.00518 (0.52% of one CPU). The stat run is more useful for overall resource demand than the sparsely sampled perf report is for attribution to individual functions. Restricted kernel addresses and absent kernel debug symbols further limit stack interpretation.

10. Task 4 - Real-time scheduling methodology

The jitter benchmark was cross-compiled for ARM and launched inside QEMU. Both tests used 1,000 absolute CLOCK_MONOTONIC deadlines, spaced by 10 ms. The experiment compared SCHED_OTHER at priority 0 with SCHED_FIFO at priority 50, verifying that the requested scheduler had been accepted before measurement.

Deadline[i+1] = Deadline[i] + 10 ms

Lateness[i] = max(0, WakeUpTime[i] - Deadline[i])

Min / Mean / Max are computed over 1,000 lateness values.

P99 is the sorted 990th observation (zero-based index 989).

The benchmark reports four bins: <10 us, 10-50 us, 50-100 us and >=100 us. This partition is coarse for a nested virtual machine: all attached observations fall into the uppermost bin, so the histogram alone cannot compare the tail distribution. The P99 and maxima provide more separation.

Measurement limitations

In the presence of prolonged emulation stalls, an absolute sleep may return immediately for several already-passed deadlines. That can increase measured lateness and make the mean much larger than the nominal 10 ms cycle. The measured values are properties of this QEMU/VirtualBox arrangement and workload, not intrinsic guarantees of the ARM processor or the SCHED_FIFO policy.

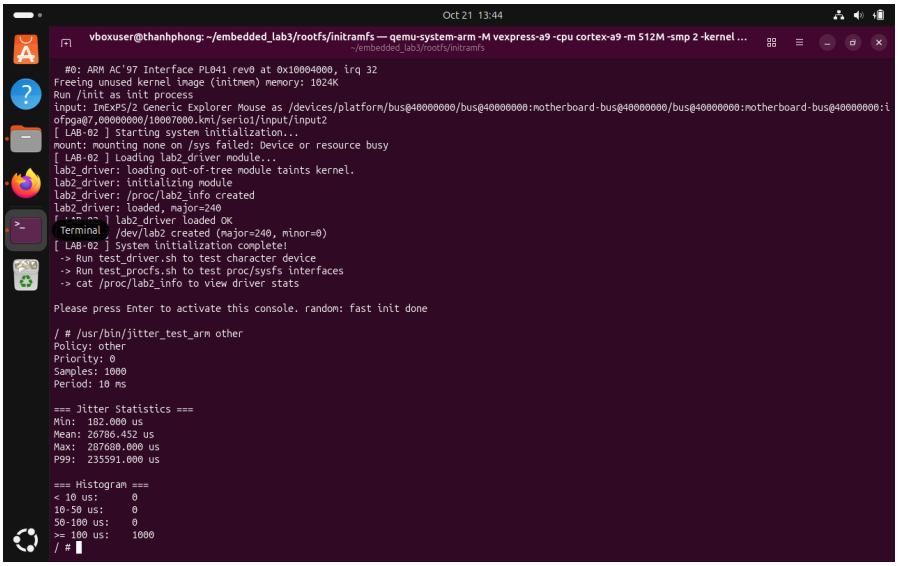


Figure 15. Scheduling test configuration and first SCHED_OTHER observations.

11. Task 4 - Experimental results

Metric	SCHED_OTHER	SCHED_FIFO
Priority	0	50
Samples	1,000	1,000
Nominal period	10 ms	10 ms
Minimum	146.000 us	136.000 us
Mean	22,620.236 us	30,683.961 us
Maximum	339,095.000 us	234,096.000 us
P99	239,976.000 us	153,596.000 us
Samples >=100 us	1,000	1,000

For the saved paired runs, SCHED_FIFO had a lower maximum and P99 lateness than SCHED_OTHER, while its mean lateness was higher. The result is mixed rather than a clean win for either policy. Other screenshot-only runs show further variability across measurements.

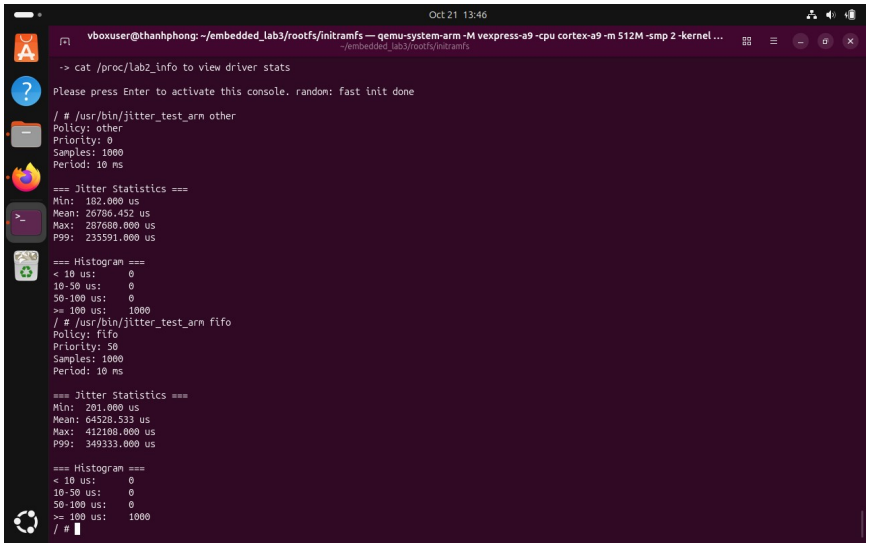


Figure 16. Earlier on-screen comparison of the two schedulers; results vary between runs.

Because sequential tests may experience different host contention, the experiment is not a controlled latency comparison. A stronger study would interleave repeated runs, capture raw timestamp series, record CPU contention, and report run-to-run dispersion and confidence intervals.

11.1 Task 4 - Trace evidence and interpretation

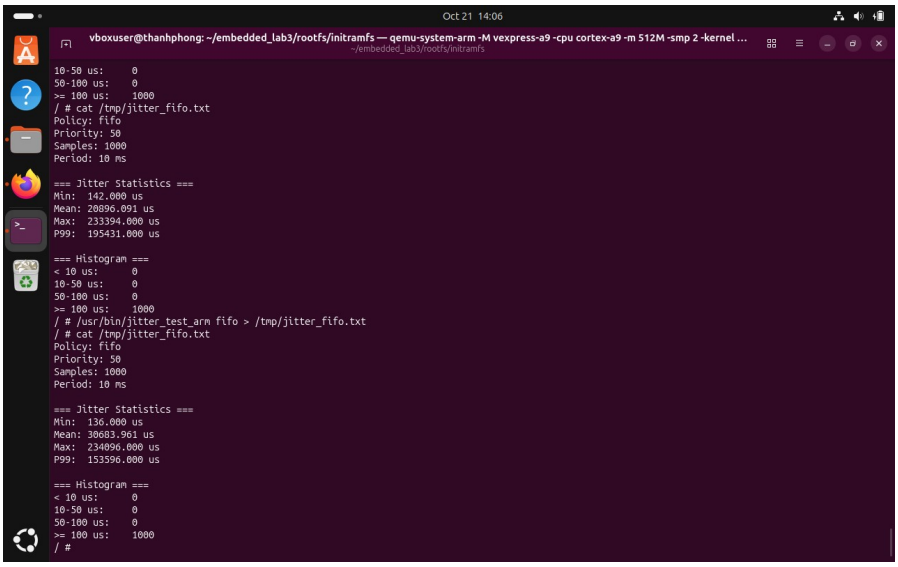


Figure 17. Saved SCHED_FIFO measurement: priority 50, 1,000 samples.

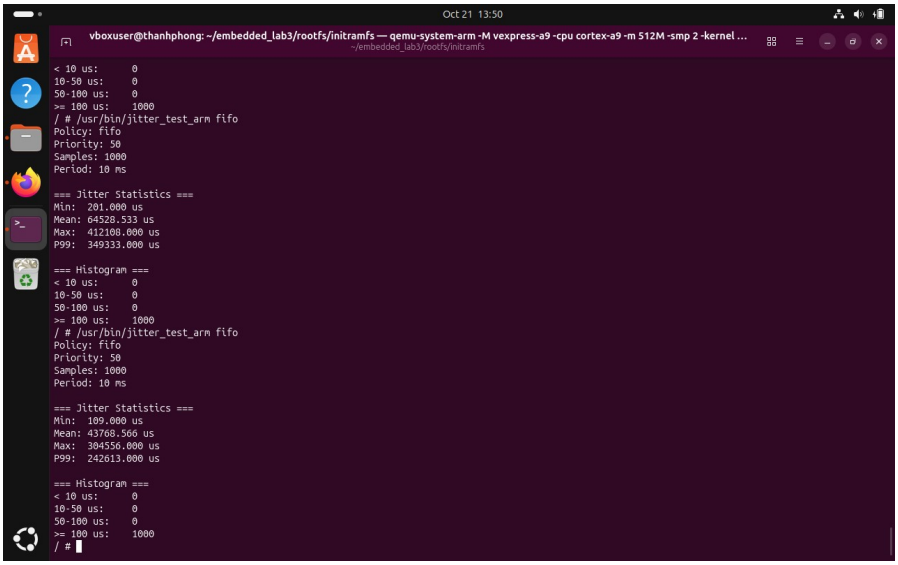


Figure 18. Repeated FIFO measurement illustrates run-to-run variability.

The uploaded task4 text files contain final summary statistics and histogram counts only. They do not preserve the 1,000 individual wake-up observations, so the percentile implementation cannot be independently recomputed from the provided archive. A follow-up version should export all raw samples with expected and actual timestamps.

12. Challenges, fixes, and residual risks

Challenge	Resolution / observation	Residual risk
Kernel message queues unavailable	Customized target kernel configuration and rebuilt boot image.	Verify when rebuilding for another board.
Sensor read timing	Track EAGAIN and successful reads; absolute timer.	Short runs may underproduce due to scheduling.
Host memory lock / RT privileges	Disable mlockall in host mock; use ARM for FIFO exercise.	Mock differs from production configuration.
Queue busy polling	Backoff nanosleep after EAGAIN.	Adds latency and not fully event driven.
Perf PMU / permissions	Use software counters and restricted profiling.	Only three record samples.
Large QEMU wake-up lateness	Present measured data and caveats faithfully.	No physical hardware validation.

Lessons learned

Test counters are useful end-to-end checks but do not replace code review and systematic fault injection. Nonblocking I/O must be matched to an explicit wait strategy, otherwise empty queues can dominate syscall counts. Real-time scheduling tests require control over the whole stack, including VM host contention. Finally, carrying screenshots, raw logs and runnable binaries alongside the report makes performance findings auditable.

Proposed follow-up tests

- Stress the queue at high sample rates, including a full buffer and failed mq_send.
- Trigger >1 MB logs and validate rotation while records are being produced.
- Inject device read, ioctl and file-open failures; inspect cleanup and exit status.
- Capture raw jitter samples over multiple randomized runs on hardware and QEMU.
- Re-test the optimized ARM executable for 60 seconds and retain driver /proc counters.

13. Conclusion

The supplied evidence supports a working multithreaded Embedded Linux monitoring pipeline on an ARM-emulated target and a reproducible host-based validation path. The application successfully coordinated sampling, processing, alerts and logging in documented runs, and its host mock exhibited stable resident memory and no Valgrind-detected leaks in the measured cases.

GDB exposed the active thread architecture and permitted breakpoint-level validation of a rolling average. strace identified an expensive queue polling pattern, and the implemented backoff reduced receive syscall volume by approximately 99.68%. perf software counters showed low average CPU use for the mock run but insufficient sampled stacks for a detailed hotspot profile. Jitter results exposed substantial emulation-related delays and differing mean versus tail behavior under SCHED_OTHER and SCHED_FIFO.

The remaining work includes broader error-path tests, raw scheduling traces, a demonstrated log-rotation stress test and the project video demonstration. These limitations are part of the engineering outcome and should be explicitly acknowledged in any assessment.

References

- Linux kernel documentation: driver interfaces, procfs and scheduling (docs.kernel.org).
- Linux manual pages: mq_overview(7), mq_timedreceive(3), clock_nanosleep(2), sched(7), pthread_setschedparam(3).
- Valgrind User Manual: Memcheck (valgrind.org).
- GDB User Manual: threads, breakpoints and backtraces (sourceware.org/gdb).
- perf tooling and perf_event security documentation (man7.org and kernel.org).
- Course assignment specification and local project source tree; verify the final submitted versions.

Appendix A. Evidence inventory

The attached evidence archive contained 22 screenshot files, plus text logs and perf.data. The table below lists the principal referenced artifacts and their role. The screenshots remain independent records and should be included in the final submission package if required by the instructor.

Artifact	Demonstrated purpose
asgn2_task1_60seconds_pass.png	ARM functional test
asgn2_task2_valgrind_30s.png	Memcheck result
memory_samples.txt	/proc RSS, VmSize, thread count
memory_maps.txt	Address space regions
gdb_backtrace.txt	Thread call stacks
strace_30s.txt / strace_30s_fixed.txt	Before / after busy polling
perf_stat.txt / perf_report.txt / perf.data	Profiling outputs
jitter_other.txt / jitter_fifo.txt	1,000-sample scheduling summaries

Evidence provenance and reproducibility

All quantitative findings on these pages are derived from uploaded logs or explicitly identified interactive observations. The screenshots are authentic evidence from the project environment. The author should cross-check the final document against the actual submission and ensure any AI-assisted drafting follows the course attribution and originality requirements.

The zip did not contain the driver source, C application sources, compiled artifacts or demonstration video. Those materials must be collected separately if required for grading. This report is not a substitute for submitting them.